Verix: Idea, Problem and Fix

Nov 3, 2026 · @Zhilo.labs

Executive summary

Verix is a proposed pull-request check, delivered first as a CI step and later as a GitHub App, that flags pull requests written by AI agents from an outdated view of the codebase, before they merge. Verdict: **validate before building**. The weighted score is 4.6/10, a decision aid and not a success probability.

- **Idea:** compare what the agent actually read and assumed against what changed on the base branch since, then say exactly which claims need re-checking.
- **Why now:** teams run several background and parallel agents, so snapshots go stale within hours. Text merges cleanly while the assumptions underneath it break.
- **Biggest doubt:** file-level staleness flagging already exists for free (projectmem), and large platforms could add it quickly. Demand and willingness to pay are unproven.
- **First move:** a public benchmark of seeded repo changes, plus 20 interviews with teams running multiple coding agents. Build no cloud product until both pass the thresholds in the validation section.

Evidence labels used below: **Verified** (sourced), **Inference** (reasoned from evidence), **Hypothesis** (needs testing).

Problem statement

AI coding agents act on beliefs they formed earlier, and nothing checks whether those beliefs still hold when the agent finally acts. The code can change underneath a plan without the agent knowing.

A concrete failure

1. 10:00: an agent reads `auth.py`, concludes authentication uses JWT, and plans refresh-token middleware.
2. 10:45: another developer or agent migrates authentication to server-side sessions.
3. 11:30: the first agent resumes, trusts its old conclusion, and opens a PR.
4. Result: the PR merges without a text conflict, may even pass tests, and encodes a wrong model of the system.

Why it is a real problem

- **Verified:** research in 2026 reports that agents trusting stale memory can do worse than agents with no memory, and practitioner posts describe stale instructions misleading coding agents (research summary, DEV post).
- **Inference:** the more agents run in parallel or in the background, the more often a plan is made on a snapshot that has since moved.
- **Hypothesis:** this causes enough rework, review time or incidents that teams will pay to prevent it. No payment evidence exists yet.

Core problem statement: when an AI agent proposes a change, nobody can see which of its assumptions about the code are no longer true, so reviewers either re-verify everything or trust blindly.

Severity, honestly: annoying and costly in long-running multi-agent work, rarely catastrophic. This is a high-value occasional problem for a niche, not yet a mission-critical one.

How teams handle it today

Existing approaches each cover part of the problem, and none targets the PR boundary with transitive checks.

Approach	What it does	Where it falls short
Re-read before edit	Agent re-opens the file before changing it	Catches file contents, not stale plans or beliefs derived from several files
Tests and CI	Fail when behavior breaks	Pass when the wrong assumption still produces working code
Human code review	Reviewer spots the mismatch	Slow, and the reviewer lacks the agent's assumptions
Memory layers (Zep, Mem0, Letta)	Store and retrieve facts, some invalidate on contradiction	Invalidate on new information, not on a source file changing
Stale-memory flags (projectmem)	Flag decisions whose cited file has new commits	File-level only, no transitive dependencies, no PR-time gate
Rebase and merge conflicts	Detect text overlap	Silent when changes do not overlap textually

Inference: the open gap is semantic: knowing which assumptions behind a PR depended on files that have since changed, even when the diffs never touch.

The solution: a stale-PR guard

Verix watches the gap between what an agent saw and what the base branch looks like now, and reports it on the pull request. It does not replace the agent, the memory layer or CI.

How it fixes the problem, step by step

1. **Record what the agent read.** A local hook logs each file the agent reads, with commit and content hash. This comes from tool calls, not from the agent declaring its own dependencies, which models under-report.
2. **Capture key claims.** Where the agent states assumptions ("auth uses JWT"), they are tied to the files and revision that support them.
3. **Detect change.** When the PR opens, or the base branch moves, compare hashes of everything the agent read against the current base.
4. **Propagate impact.** Files that changed make the claims built on them *suspect*. Decisions and planned edits that depend on those claims become suspect too.
5. **Report on the PR.** A comment lists the stale reads, the affected claims, and the smallest checks to resolve them (diff this file, run that test).
6. **Gate when risk is high.** On opt-in policy, a status check fails until the agent re-verifies. The result is revision-bound, so a later change reopens it.

Four claim states. *Valid*: evidence matches current code. *Suspect*: a relevant file changed, verification needed. *Invalid*: new evidence contradicts it. *Unknown*: not enough evidence to say. A changed file makes a claim uncertain, not false.

What it will not claim: that it proves a PR is correct. It reports which assumptions are no longer supported by current evidence, and says plainly when its dependency view is incomplete.

Example PR comment (illustrative)

```
Verix: 2 of 5 assumptions need re-checking
- Suspect: "Auth uses JWT" (read app/auth.py @ abc123; changed in def456)
- Suspect: "Refresh middleware is unused" (depends on the claim above)
Suggested checks: diff app/auth.py abc123..def456, run tests/test_auth.py
Status: held until re-verified
```

How users run it: an operator installs Verix locally. Its hooks (Claude Code first, Codex once its hooks feature is confirmed) log what the agent reads and can hold risky edits. A small session receipt (paths, hashes, claims, no file bodies) travels with the pull request. A CI step first, and later a GitHub App, compares the receipt with the base branch and posts the comment above. Details are in the PRD, under Usage modes.

Technical design and feasibility

The core is deterministic. An LLM is used only where meaning must be interpreted, and never as the authority on whether something is safe.

Components, in data-flow order

1. **Capture hook:** local, inside the agent harness (Claude Code hooks first). Logs reads, writes and stated claims with commit and content hash.
2. **Change detector:** Git webhooks and hash comparison against the base branch.
3. **Evidence graph:** claims, observations, decisions and planned edits as nodes, justification links as edges. Start in PostgreSQL tables, not a graph database.
4. **Impact engine:** reachability over the graph marks affected nodes *suspect*, then looks for independent supporting evidence before escalating.
5. **Verification planner:** picks the cheapest checks: re-read a file, compare a symbol, run a test, or ask a human.
6. **Policy engine:** deterministic rules turn results into allow, verify, or block, with a reason and the resolving checks.
7. **GitHub App:** posts the PR comment and sets the status check.

Deterministic vs LLM

Task	Method
Change detection, hashing, graph traversal, policy	Deterministic
Extracting claims from agent text	LLM plus validation
Reading ambiguous code changes	LLM-assisted
Explaining a flagged assumption	LLM, grounded in the diff

Key formula. For changed sources C in dependency graph G, the first-pass affected set is the nodes reachable from C. Reachable means *potentially* affected, never *proven* invalid.

```
I(C) = \operatorname{Reachable}(C, G)
```

Feasibility, separated into four levels

- **Technically possible:** yes. Hooks, hashes and a PR App are standard.
- **Practical:** likely for Claude Code first. Other agents differ in how much they expose, which limits coverage.
- **Reliable in production:** unproven. Hidden dependencies (config, runtime behavior, external services) will cause misses.

- **Economically sustainable:** probably, since most work is non-LLM and cheap.

Hard problems

- Incomplete graphs: when dependencies are unknown and risk is high, return "unresolved" rather than a false all-clear.
- Race condition: the repo can change after verification but before merge. Bind approvals to a revision and recheck at merge.
- Enforcement limits: a hook is not a security boundary. A call that bypasses it bypasses the guard. Strict enforcement needs a gateway.
- Privacy: stay local-first and send only hashes and metadata by default, never whole private repositories.

The Claude Code `PreToolUse` hook can block a tool call, so in-session enforcement is feasible (docs).

Market and competitors

Memory infrastructure is crowded and commoditizing, and the stale-memory problem is already named by competitors. Researched Nov 3, 2026 from search results and vendor pages; I did not open every page in full.

Competitor	What it does (verified)	Gap for Verix
Zep / Graphiti	Temporal context graph, fact validity windows, invalidation, provenance to source episodes	Invalidates on contradicting new data, not on a source file changing; no PR gate
projectmem v0.1.4	Compares recorded decisions to git history of the cited file; flags rather than deletes	File-level only; no transitive dependencies; no PR check. Closest direct competitor
agentmemory	MCP memory for many coding agents; provenance, audit, multi-agent leases, checkpoint gates	General memory; not a PR-time stale-assumption report
Mem0, Letta	Extract and retrieve memory; Letta adds an agent runtime	No change-driven invalidation tied to repo state
Claude Code hooks	Lifecycle hooks that can block tool calls (docs)	A platform you build on, and a platform that could add this itself

Pricing anchors (verified, 2026): Mem0 from free to $19, $79 and $249 per month; Zep Flex $125 per month; Letta Pro $20 per month (comparison). Mem0 raised $24M in Oct 2025 (Sacra). Zep retired self-hosted Community Edition and moved to credit pools. AWS

and Google began billing for bundled agent memory in Sep and Oct 2026.

Inference: paying customers for "memory" will compare you to open source and to cloud bundles. A stale-PR guard sits one layer up, at review, where I found no direct product.

Market size: not established. I found no reliable count of teams running multiple coding agents in parallel, so I will not publish TAM, SAM or SOM. The bottom-up model to build from interviews is: teams running 3 or more concurrent agents × share that hit costly stale incidents × price. Every input is currently unknown.

Platform risk: GitHub, Anthropic, Cursor or an agent-memory vendor could ship a basic version in weeks. That is the largest existential risk, covered in the risks section.

MVP and roadmap

The first release is a benchmark and a local tool, not a platform. One customer segment, one failure, one measurable result.

MVP (validation version, indicative weeks 1 to 6)

- **Segment:** small teams running 3 or more concurrent coding agents on a shared repository, starting with Claude Code users.
- **Workflow:** agent works, repo changes, agent opens a PR, Verix reports stale assumptions.
- **Build:** open-source CLI and Claude Code hook; read and hash logging; explicit claims with source files; transitive suspect marking; allow, verify or block output; a local report. A basic GitHub comment is optional.
- **Exclude:** cloud dashboard, multi-agent coordination, automatic claim extraction, enterprise features, other agent frameworks, any LLM in the enforcement path.
- **Release criteria:** passes the benchmark thresholds in the validation section on at least 30 seeded scenarios.

Roadmap, gated by evidence

1. **V0: benchmark and local guard.** Gate: recall of at least 80% and false blocks under 10%, clearly better than file-hash flagging alone.
2. **V1: CI step and GitHub App.** PR comments and status checks, repo-level policies, hosted history. Gate: 3 design partners using it on real repos, at least one paying or committed in writing.
3. **V2: expansion.** Multi-agent visibility, Cursor and Codex integrations, selective re-verification, TypeScript SDK. Build only features that paying customers asked for.
4. **V3: scale.** SSO, self-hosted deployment, audit exports, a verification API for teams embedding it in their own agent platforms.

Priority table for V0 and V1

Feature	Who needs it	Complexity	Success metric
Read and hash capture hook	Every user	Low	Capture coverage of agent reads
Suspect propagation	Teams with plans spanning files	Medium	Invalidation recall and precision
PR comment with checks	Reviewers	Low to medium	Share of flags acted on
Status-check gate	Teams with strict merge rules	Medium	False-block rate, bypass rate
Benchmark harness	You, and the community	Medium	Reproducible public results

Business model and go-to-market

Charge for verification and review outcomes, not for stored memories. Memory storage is priced against free and open-source options.

Pricing: all hypotheses, none validated

Plan	Price	Includes
Open source	$0	Local CLI and hook, one repository, local reports
Pro	$29 per month	Hosted history, GitHub comments, one team's repos, usage allowance
Team	$149 per month	Shared policies, status-check gates, multi-repo visibility, higher limits
Enterprise	Custom	Self-hosting, SSO, audit export, SLA

Arithmetic only, not a forecast: 100 Team customers at $149 is $14,900 MRR, or $178,800 ARR. Unit costs are likely low because the core is non-LLM, but I have no usage data, so I have not modeled margin, churn or break-even.

Funding path: bootstrap, or accelerator-style capital, for now. Venture scale depends on the verification API becoming embedded in other agent platforms, which is a thesis and not evidence.

Go-to-market, by stage

- **First 10 users:** personal outreach to teams building or heavily using coding agents; free audit of one of their repos with consent.
- **First 100 users:** open-source release, the benchmark write-up, a Hacker News post, and a short demo of two agents colliding on one repo.
- **First 1,000 users:** GitHub Marketplace listing, Claude Code and MCP ecosystem posts, technical content on stale agent context.
- **First 100 paying customers:** convert teams who rely on PR comments daily. Do not assume virality: distribution here is the PR comment itself, seen by every reviewer.

Customer discovery question to ask first: "When an agent's PR is based on a snapshot that has since changed, how do you find out, and what was the last time it cost you time?" Ask for a dated incident, hours lost, and the safeguard they use now. "It never happens" is useful negative evidence.

Risks, validation and 90-day plan

The strongest argument against this product is that it becomes a feature. The experiments below are built to test that first.

Risks

Risk	Likelihood	Impact	Mitigation and test
Becomes a feature of GitHub, an agent vendor or a memory product (existential)	High	High	Win on a public benchmark; test whether partners pay beyond free tools
Incomplete dependency view gives false confidence	High	Medium	Infer from tool-call reads; return "unresolved" when unsure; seeded hidden-dependency scenarios
Stale-PR incidents are too rare or cheap to pay for	Medium	High	20 interviews asking for dated incidents and hours lost
False blocks make developers disable it	Medium	Medium	Advisory mode by default; measure false-block rate and bypass rate
Founder capacity: team of 3, an agency and college	High	Medium	One narrow release; no cloud build before the gates pass

Scorecard (preliminary judgment, not statistical): 4.6 out of 10

Dimension	Weight	Score	Basis
Problem severity	15%	6	Sourced complaints, no cost data
Customer demand evidence	15%	3	None direct
Willingness to pay	10%	3	None
Competitive differentiation	10%	5	Gap exists, narrow
Market accessibility	10%	5	Reachable via GitHub
Technical feasibility	10%	6	Standard parts
Distribution potential	10%	5	PR comment is a channel
Retention potential	10%	4	Depends on incident rate
Unit economics	5%	5	Mostly non-LLM
Defensibility	5%	3	Thin without benchmark lead

Experiments, with decision thresholds

Experiment	Procedure	Proceed if	Stop if
Seeded benchmark	30 scenarios: rename, auth change, schema change, dependency bump, hidden dependency; compare against file-hash flagging and projectmem	Recall 80% or higher, false blocks under 10%, clear gap over file-hash baseline	File-hash flagging catches 75% or more of what the graph does
Problem interviews	20 teams running multiple coding agents	At least 6 describe a costly incident in the last 60 days	Fewer than 3 do
Paid pilot	3 design partners on real repos	At least 1 pays $500 or more, or commits in writing	No one pays or commits in 90 days

Metrics: invalidation recall = TP / (TP + FN); also precision, unsafe-action rate, task completion, verification overhead, recovery success, false-block rate.

30, 60 and 90 days

- **Days 1 to 30:** build the benchmark harness and the minimal Claude Code hook. Run 10 interviews. Write the benchmark protocol before running agents.
- **Days 31 to 60:** publish results and the open-source release. Finish 20 interviews. Approach 20 teams for design partners.
- **Days 61 to 90:** pilots with 3 partners, a basic GitHub comment, and a pricing test. Decide: build, pivot or stop, using the thresholds above.

Fallback: if the product thresholds fail, the benchmark and a short paper on stale-assumption failures in agent PRs still stand as research output.

Not verified: claims about LangGraph, Temporal, LangSmith, the OpenAI Agents SDK and "Squish" in the original concept doc, and any demand or market-size figure.

Sources: Graphiti README · projectmem v0.1.4 · agentmemory · Claude Code hooks · Memory pricing comparison · Mem0 funding · Agent memory research · Stale context post